Trabalho Prático
Deep Q-Network no Lunar Lander

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

Este trabalho aborda o problema **Lunar Lander**, disponibilizado pela biblioteca Gymnasium. O objetivo do ambiente é controlar os motores de uma nave espacial para realizar um pouso suave em uma zona delimitada na superfície lunar, lidando com a física de gravidade e inércia.

O ambiente apresenta as seguintes características principais:

- **Espaço de Estados:** Contínuo, composto por um vetor de 8 dimensões que inclui coordenadas (x, y), velocidades lineares, ângulo, velocidade angular e indicadores de contato das pernas com o solo.
- **Espaço de Ações:** Discreto, com 4 opções de controle: não fazer nada, acionar o motor principal, acionar o motor esquerdo ou acionar o motor direito.
- **Recompensa:** Baseada na proximidade do alvo, suavidade do pouso e economia de combustível (penalidade por uso dos motores).

A solução proposta consiste na implementação do algoritmo **Deep Q-Network (DQN)**. Esta técnica estende o algoritmo Q-Learning tradicional utilizando Redes Neurais Profundas como aproximadores de função para estimar os valores Q(s, a), permitindo lidar eficientemente com o espaço de estados contínuo do problema.

O desenvolvimento do trabalho foi estruturado nas seguintes etapas:

- Implementação manual do agente DQN utilizando a biblioteca PyTorch, incorporando mecanismos de Experience Replay e Target Networks com atualização periódica para estabilidade do aprendizado.
- Realização de uma busca sistemática de hiperparâmetros para identificar a configuração que maximiza o retorno médio e a estabilidade do agente.
- Análise comparativa de desempenho entre a solução implementada e a versão de referência da biblioteca Stable Baselines3 (SB3).

2. Implementação

A solução foi desenvolvida utilizando a linguagem Python, estruturada em um Jupyter Notebook para facilitar a experimentação iterativa, a reprodutibilidade e a visualização dos resultados. O código foi organizado de forma modular, separando as definições de arquitetura, o agente, o loop de treinamento, a busca de hiperparâmetros e a comparação com Stable Baseline3.

Tecnologias e Bibliotecas Utilizadas

- **Gymnasium:** Utilizada para instanciar e interagir com o ambiente LunarLander-v3. Esta biblioteca fornece a interface padrão para observação de estados e execução de ações, além de gerenciar a física do simulador Box2D.
- **PyTorch:** Framework de aprendizado profundo escolhido para a construção e treinamento das redes neurais devido à sua eficiência computacional (suporte a CUDA/GPU) e flexibilidade na diferenciação automática (autograd).
- **NumPy:** Empregada para manipulação eficiente de arrays, operações matemáticas vetoriais e gerenciamento de buffers de memória.
- **Stable Baselines3:** Utilizada exclusivamente na etapa final de comparação para fornecer uma implementação de referência do algoritmo DQN.
- **Ferramentas Auxiliares:** pickle e os para gerenciamento de checkpoints e persistência dos dados gerados durante a busca de hiperparâmetros. itertools para geração eficiente da grade de testes. E matplotlib/pandas para visualização e análise de dados.

Arquitetura do Agente DQN

A implementação do agente seguiu a arquitetura clássica do Deep Q-Network (DQN), adaptada para as especificidades do ambiente.

a. Aproximador de Função (Rede Neural):

Foi implementada uma rede neural feedforward (Perceptron Multicamadas) para aproximar a função de valor Q(s, a).

- **Arquitetura:** A rede é composta por uma camada de entrada compatível com o vetor de estado (8 dimensões), seguida por duas camadas ocultas lineares com 256 neurônios cada, utilizando a função de ativação ReLU. A camada de saída possui 4 neurônios lineares, correspondendo aos valores-Q das ações possíveis.
- **Otimização:** O treinamento utilizou o otimizador Adam com a função de perda de Erro Quadrático Médio (MSE Loss).

b. Buffer de Replay (Experience Replay):

Para mitigar a correlação temporal entre amostras consecutivas e estabilizar o treinamento, utilizou-se um buffer circular que armazena as transições (s, a, r, s', done). Durante o treinamento, um minibatch aleatório é

amostrado para quebrar a correlação temporal entre os dados, permitindo um aprendizado mais estável.

c. **Redes Alvo (Target Networks):**

Para evitar a oscilação dos valores alvo durante o treinamento, o agente mantém duas redes idênticas: a rede local (treinada a cada passo) e a rede alvo (usada para calcular o erro TD). A atualização da rede alvo foi implementada através da estratégia de Hard Update: em intervalos fixos de passos (definidos pelo hiperparâmetro target_update_freq), os pesos da rede alvo são substituídos integralmente por uma cópia exata dos pesos da rede local. Essa abordagem garante que os valores alvo permaneçam estacionários por períodos determinados, aumentando a estabilidade da convergência.

Decisões de Implementação e Busca de Hiperparâmetros

Para garantir a robustez da solução, foram adotadas as seguintes estratégias de implementação:

- **Política epsilon-greedy com Decaimento:** A exploração foi gerenciada por uma política epsilon-greedy onde o valor de epsilon decai exponencialmente a cada passo da simulação, iniciando em 1.0 (exploração total) até um valor mínimo de 0.01 (conforme definido nos parâmetros fixos dos experimentos).
- **Estabilização de Gradientes:** Foi aplicada a técnica de Gradient Clipping, limitando a norma do gradiente a 10 durante o processo de backpropagation. Essa medida preventiva evita o problema de explosão de gradientes (exploding gradients), comum em treinamentos de Aprendizado por Reforço Profundo onde as atualizações podem ser ruidosas.
- **Grid Search com Checkpointing:** A busca de hiperparâmetros foi implementada de forma automatizada para testar 96 combinações distintas de parâmetros (Taxa de Aprendizado, Fator de Desconto, Tamanho do Buffer, Tamanho do Batch, Frequência de Atualização e Decaimento de Epsilon). Foi incluído um mecanismo de persistência de dados (checkpointing com pickle) para salvar resultados parciais em disco, permitindo a execução dos experimentos de longa duração sem risco de perda de dados.
- **Critério de Seleção:** A escolha do melhor modelo foi baseada na estabilidade de longo prazo, calculada pela média de retorno dos últimos 200 episódios em um treinamento de 1000 episódios.

3. Avaliação Experimental

Para validar a eficácia da solução proposta e garantir a reprodutibilidade dos resultados, foram conduzidos experimentos em um ambiente controlado. O hardware utilizado para o treinamento incluiu uma GPU NVIDIA GeForce RTX 2060, o que acelerou significativamente o processamento das redes neurais.

Metodologia de Busca de Hiperparâmetros

A definição dos hiperparâmetros é crítica para o sucesso do DQN, visto que o algoritmo é sensível a instabilidades. Adotou-se a estratégia de **Grid Search** (Busca em Grade), totalizando **96 configurações experimentais** distintas geradas pelo produto cartesiano das seguintes variáveis:

- **Taxa de Aprendizado (Learning Rate):** {1e-3, 5e-4}, 1e-4}
- **Fator de Desconto (gamma):** {0.99, 0.95}
- **Tamanho do Replay Buffer:** {50.000, 100.000}
- **Tamanho do Batch:** {64, 128}
- **Frequência de Atualização da Rede Alvo:** {500, 1.000} passos
- **Decaimento do Epsilon:** {0.995, 0.990} aplicado a cada passo.

Para cada experimento, o agente foi treinado por 1.000 episódios. O critério de seleção para o melhor modelo foi a **média de retorno dos últimos 200 episódios**, uma métrica que privilegia a estabilidade de convergência a longo prazo em detrimento de picos isolados de performance.

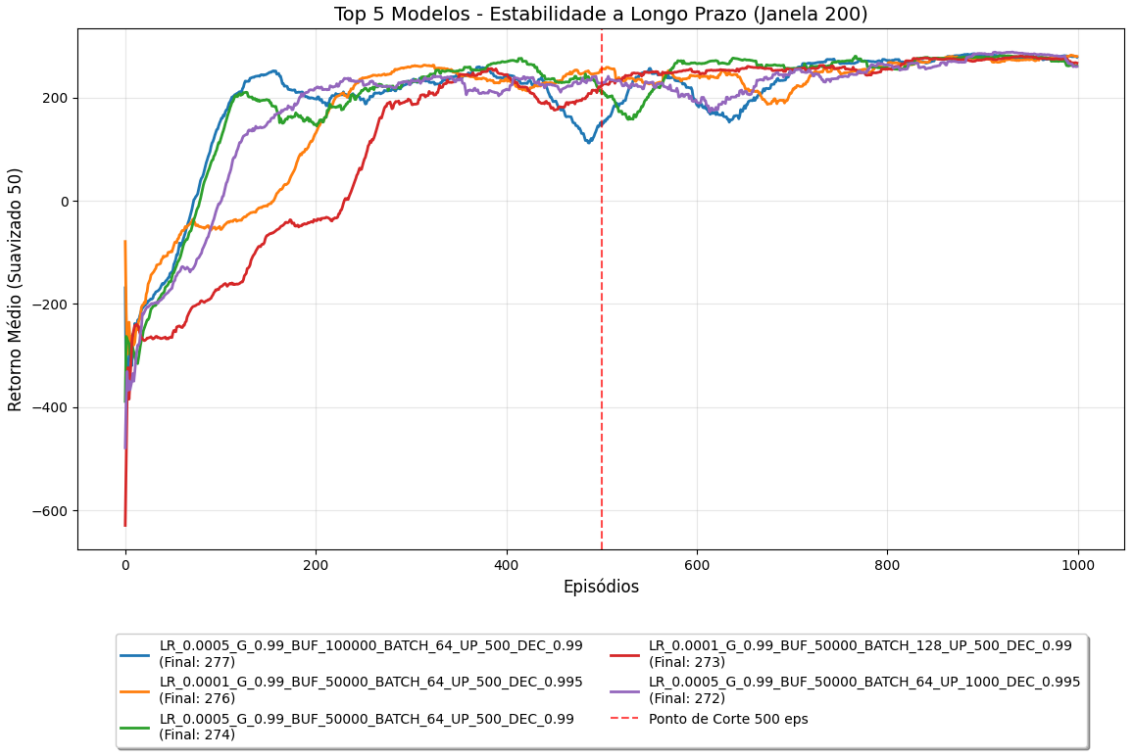
Resultados da Otimização

A análise dos 96 experimentos revelou que o agente é particularmente sensível à taxa de aprendizado e ao tamanho do buffer. A configuração vencedora obteve um score médio final de **277.10**, demonstrando uma solução robusta para o problema (que é considerado resolvido com 200 pontos).

A melhor configuração de hiperparâmetros identificada foi:

- **Learning Rate:** 5e-4 (0.0005) — Valores maiores causaram divergência e menores tornaram o aprendizado muito lento.
- **Gamma:** 0.99 — Essencial para valorizar recompensas futuras (pouso).
- **Replay Buffer:** 100.000 — Um buffer maior permitiu maior diversidade de amostras, reduzindo o esquecimento.
- **Batch Size:** 64
- **Target Update:** 500 passos
- **Epsilon Decay:** 0.99

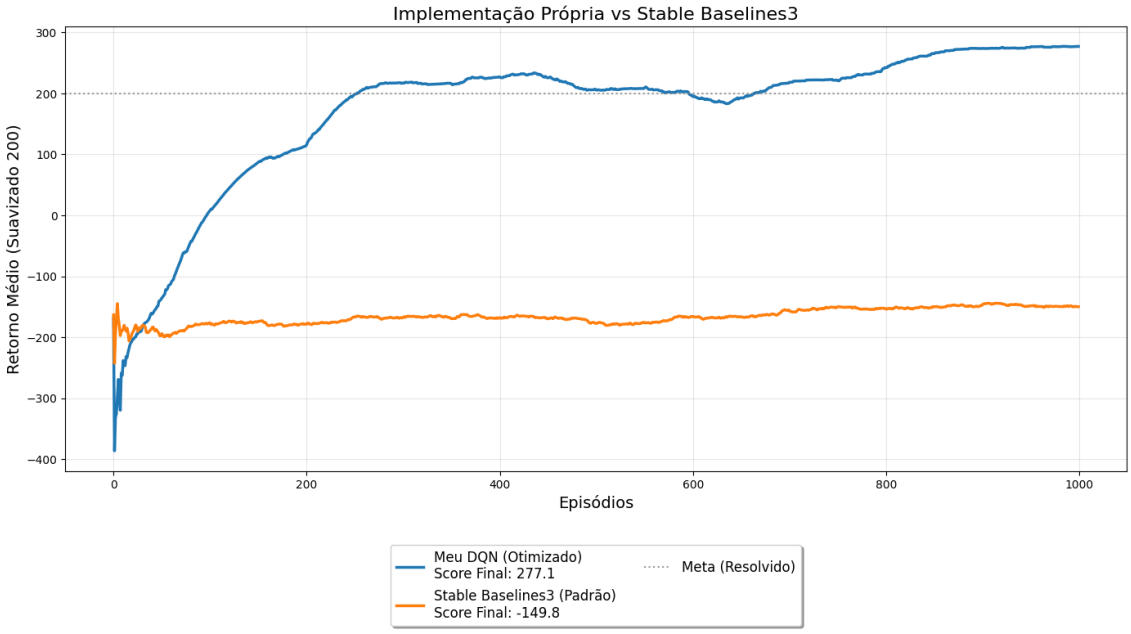
O resultado pode ser verificado na imagem abaixo. A legenda é ordenada do melhor para o pior.



Análise Comparativa: Implementação Própria vs. Stable Baselines3

Após a identificação dos melhores hiperparâmetros, o desempenho do agente implementado foi comparado com a implementação de referência da biblioteca Stable Baselines3 (SB3), utilizando os hiperparâmetros padrão (default) da biblioteca. Ambos os agentes foram submetidos ao mesmo protocolo de treinamento de 1.000 episódios com a semente aleatória fixada (SEED=33) para garantir uma comparação justa.

Os resultados obtidos foram:



Métrica	DQN Implementado	Stable Baselines3
Média Final	277.1	-143.9
Score Máximo Atingido	322.5	74.8
Status do Ambiente	Resolvido	Não Resolvido

Discussão dos Resultados:

A implementação própria apresentou uma grande superioridade sobre a versão padrão do SB3. Essa diferença não indica falha no algoritmo da biblioteca de referência, mas sim a importância da otimização de hiperparâmetros para o contexto específico do problema.

A causa principal da divergência foi identificada na configuração padrão do parâmetro learning_starts do SB3, que define um período de aquecimento de 50.000 passos. Durante essa fase, o agente apenas coleta transições para o replay buffer sem realizar atualizações na rede neural.

Analisando os logs de execução:

a. Dado que o agente não treinado tende a encerrar episódios rapidamente por colisão ou instabilidade, a média observada foi de aproximadamente 96 passos por episódio no SB3.
b. Consequentemente, a execução total dos 1.000 episódios gerou aproximadamente apenas 96.000 passos totais.
c. Isso significa que o agente do Stable Baselines3 passou mais de 52% do tempo total do experimento (os primeiros 50 mil passos) sem realizar qualquer aprendizado (backpropagation).

Por outro lado, a solução implementada foi configurada para iniciar a atualização dos pesos imediatamente após o preenchimento do primeiro lote (batch_size = 64). Isso permitiu que o agente aproveitasse a toda experiência disponível, garantindo atualizações de gradiente desde o início do treinamento. Essa decisã, aliada ao uso de um buffer maior para evitar overfitting em dados recentes, foi determinante para que a solução manual atingisse a convergência enquanto a referência permaneceu estagnada.

4. Conclusão

O trabalho atingiu os objetivos propostos, resultando na implementação de um agente Deep Q-Network (DQN) capaz de solucionar o ambiente LunarLander-v3. A solução desenvolvida, utilizando PyTorch convergiu para uma política ótima, e demonstrou estabilidade e robustez ao lidar com o espaço de estados contínuo e a física do ambiente.

Os resultados experimentais evidenciaram que o sucesso do DQN é fortemente dependente da seleção dos hiperparâmetros. A busca sistemática revelou que um buffer de repetição maior (100.000 transições) e uma taxa de aprendizado moderada (5e-4) foram responsáveis pela estabilidade do aprendizado. Com essa configuração, o agente alcançou uma recompensa

média de 277.1 nos últimos episódios, superando com folga o critério de 200 pontos que define o ambiente como resolvido.

A análise comparativa destacou uma diferença significativa de desempenho em relação à implementação padrão da biblioteca Stable Baselines3 (SB3). Enquanto a solução própria convergiu rapidamente, o agente SB3 falhou em aprender a tarefa dentro dos 1.000 episódios disponíveis. Foi identificado que isso ocorreu por conta da configuração conservadora de aquecimento (warm-up) do SB3, mostrando que algoritmos de Aprendizado por Reforço Profundo exigem adaptação específica para cada domínio, e que soluções genéricas podem não ser eficazes em cenários com restrição de tempo de treinamento.

Trabalhos Futuros e Melhorias

Para aprimoramentos futuros e continuidade deste trabalho, as seguintes direções podem ser tomadas:

a. **Variações do Algoritmo:** Implementar melhorias conhecidas do DQN, como Double DQN (para reduzir a superestimação dos valores Q) e Dueling DQN (para separar a estimativa de valor do estado da vantagem da ação), avaliando se trazem ganhos de estabilidade ou eficiência.
b. **Prioritized Experience Replay:** Substituir a amostragem uniforme do buffer de replay por uma amostragem priorizada, focando nas transições onde o agente cometeu maiores erros de predição (maior erro TD), o que teoricamente acelera a convergência.
c. **Ajuste Fino do SB3:** Realizar uma busca de hiperparâmetros também para o agente do Stable Baselines3, permitindo uma comparação mais justa entre a melhor versão de ambas as implementações.
d. **Generalização:** Testar a robustez do agente treinado em variações do ambiente, como alterações na gravidade ou turbulência de vento, para avaliar a capacidade de generalização da política aprendida.
e. **Busca de hiperparâmetros:** Expandir a busca de hiperparâmetros no DQN implementado, utilizando mais variações para cada parâmetro. Importante ter tempo e recurso disponível, as configurações atuais gastaram aproximadamente 3 dias e meio para rodar as 96 simulações.